

Screens Reference

Complete reference for every screen in the b-secure user app with TypeScript component implementations.

Table of Contents

1. [Login Screen](#)
 2. [OTP Verify Screen](#)
 3. [Home Screen](#)
 4. [Guard Profile Screen](#)
 5. [Create Booking Screen](#)
 6. [Booking Tracking Screen](#)
 7. [Booking Detail Screen](#)
 8. [Bookings Tab](#)
 9. [Wallet Tab](#)
 10. [Profile Tab](#)
 11. [SOS Screen](#)
-

1. Login Screen

app/(auth)/login.tsx

```
import { useState } from 'react';
import {
  View,
  Text,
  TextInput,
  TouchableOpacity,
  KeyboardAvoidingView,
  Platform,
  ActivityIndicator,
  Alert,
} from 'react-native';
import { router } from 'expo-router';
import { useOTP } from '@/hooks/useOTP';
import { StatusBar } from 'expo-status-bar';

const COUNTRY_CODE = '+91';
```

```

export default function LoginScreen() {
  const [phone, setPhone] = useState('');
  const { requestOTP, isRequestingOTP } = useOTP(phone);

  const isValid = phone.replace(/\D/g, '').length === 10;

  const handleSendOTP = async () => {
    if (!isValid) return;
    try {
      await requestOTP();
      router.push({
        pathname: '/(auth)/otp-verify',
        params: { phone, countryCode: COUNTRY_CODE },
      });
    } catch (err: any) {
      Alert.alert('Error', err?.message ?? 'Failed to send OTP.
Try again.');
```

```

<View className="w-px h-6 bg-slate-500 mr-3" />
<TextInput
  className="flex-1 text-white text-base"
  placeholder="10-digit mobile number"
  placeholderTextColor="#64748b"
  keyboardType="phone-pad"
  maxLength={10}
  value={phone}
  onChangeText={setPhone}
  returnKeyType="done"
  onSubmitEditing={handleSendOTP}
  autoFocus
/>
</View>

<TouchableOpacity
  onPress={handleSendOTP}
  disabled={!isValid || isRequestingOTP}
  className={`h-14 rounded-xl items-center justify-
center ${
  isValid && !isRequestingOTP ? 'bg-blue-600' : 'bg-
slate-600'
}`}
  activeOpacity={0.8}
>
  {isRequestingOTP ? (
    <ActivityIndicator color="#fff" />
  ) : (
    <Text className="text-white font-semibold text-
base">
      Send OTP
    </Text>
  )}
</TouchableOpacity>
</View>

<Text className="text-slate-500 text-xs text-center mt-6
leading-5">
  By continuing you agree to our{' '}
  <Text className="text-blue-400">Terms of Service</Text>
  and{' '}
  <Text className="text-blue-400">Privacy Policy</Text>
</Text>
</View>
</KeyboardAvoidingView>
);
}

```

2. OTP Verify Screen

app/(auth)/otp-verify.tsx

```
import { useState, useEffect, useRef } from 'react';
import {
  View,
  Text,
  TouchableOpacity,
  KeyboardAvoidingView,
  Platform,
  ActivityIndicator,
  Alert,
} from 'react-native';
import { router, useLocalSearchParams } from 'expo-router';
import * as SecureStore from 'expo-secure-store';
import { OTPInput } from '@components/OTPInput';
import { useOTP } from '@hooks/useOTP';
import { useAuthStore } from '@store/useAuthStore';

const OTP_LENGTH = 6;
const RESEND_COOLDOWN = 60;

export default function OTPVerifyScreen() {
  const { phone, countryCode } = useLocalSearchParams<{
    phone: string;
    countryCode: string;
 }>();

  const [otp, setOtp] = useState('');
  const [countdown, setCountdown] = useState(RESEND_COOLDOWN);
  const timerRef = useRef<ReturnType<typeof setInterval> | null>(null);

  const { verifyOTP, requestOTP, isVerifyingOTP, isRequestingOTP } =
    useOTP(phone ?? '');
  const login = useAuthStore((s) => s.login);

  // Countdown timer
  useEffect(() => {
    timerRef.current = setInterval(() => {
      setCountdown((prev) => {
        if (prev <= 1) {
          clearInterval(timerRef.current!);
          return 0;
        }
        return prev - 1;
      });
    }, 1000);
    return () => clearInterval(timerRef.current!);
  }, []);
```

```

const resetCountdown = () => {
  clearInterval(timerRef.current!);
  setCountdown(RESEND_COOLDOWN);
  timerRef.current = setInterval(() => {
    setCountdown((prev) => {
      if (prev <= 1) {
        clearInterval(timerRef.current!);
        return 0;
      }
      return prev - 1;
    });
  }, 1000);
};

// Auto-submit when all 6 digits entered
useEffect(() => {
  if (otp.length === OTP_LENGTH) {
    handleVerify(otp);
  }
}, [otp]);

const handleVerify = async (code: string) => {
  try {
    const result = await verifyOTP(code, countryCode ?? '+91');
    await login(result.access, result.refresh, result.user);
    router.replace('/(tabs)/home');
  } catch (err: any) {
    Alert.alert('Invalid OTP', err?.message ?? 'Please try
again. ');
    setOtp('');
  }
};

const handleResend = async () => {
  if (countdown > 0) return;
  try {
    await requestOTP();
    resetCountdown();
  } catch (err: any) {
    Alert.alert('Error', err?.message ?? 'Could not resend
OTP. ');
  }
};

return (
  <KeyboardAvoidingView
    behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
    className="flex-1 bg-slate-900"
  >
    <View className="flex-1 justify-center px-6">
      <TouchableOpacity
        onPress={() => router.back()}
        className="absolute top-14 left-6"
      >

```

```

        <Text className="text-blue-400 text-base">← Back</Text>
    </TouchableOpacity>

    <View className="items-center mb-10">
        <Text className="text-white text-3xl font-bold mb-2">
            Verify OTP
        </Text>
        <Text className="text-slate-400 text-sm text-center
leading-5">
            We sent a 6-digit code to{'\n'}
            <Text className="text-white font-medium">
                {countryCode} {phone}
            </Text>
        </Text>
    </View>

    <OTPInput
        length={OTP_LENGTH}
        value={otp}
        onChange={setOtp}
        disabled={isVerifyingOTP}
    />

    {isVerifyingOTP && (
        <View className="items-center mt-6">
            <ActivityIndicator color="#3b82f6" />
            <Text className="text-slate-400 mt-2 text-
sm">Verifying...</Text>
        </View>
    )}

    {/* Resend */}
    <View className="items-center mt-8">
        {countdown > 0 ? (
            <Text className="text-slate-400 text-sm">
                Resend OTP in{' '}
            <Text className="text-white font-medium">{countdown}
s</Text>
            </Text>
        ) : (
            <TouchableOpacity onPress={handleResend}
disabled={isRequestingOTP}>
                <Text className="text-blue-400 text-sm font-medium">
                    {isRequestingOTP ? 'Sending...' : 'Resend OTP'}
                </Text>
            </TouchableOpacity>
        )}
    </View>
</View>
</KeyboardAvoidingView>
);
}

```

3. Home Screen

app/(tabs)/home.tsx

```
import { useRef, useMemo, useCallback } from 'react';
import {
  View,
  Text,
  TouchableOpacity,
  ActivityIndicator,
  Dimensions,
} from 'react-native';
import MapView, { Marker, PROVIDER_GOOGLE, Region } from 'react-native-maps';
import { router } from 'expo-router';
import BottomSheet, { BottomSheetScrollView } from '@gorhom/bottom-sheet';
import { Shield, Clock, MapPin } from 'lucide-react-native';
import { useLocation } from '@/hooks/useLocation';
import { useNearbyGuards } from '@/hooks/useNearbyGuards';
import { useBookingStore } from '@/store/useBookingStore';
import { MapGuardMarker } from '@/components/MapGuardMarker';
import { GuardCard } from '@/components/GuardCard';
import { ENV } from '@/lib/constants';
import type { Guard } from '@/types';

const { height: SCREEN_HEIGHT } = Dimensions.get('window');

export default function HomeScreen() {
  const mapRef = useRef<MapView>(null);
  const bottomSheetRef = useRef<BottomSheet>(null);
  const snapPoints = useMemo(() => ['25%', '50%', '85%'], []);

  const { currentLocation, permissionStatus } = useLocation();
  const { activeBooking } = useBookingStore();

  const { data: nearbyGuards, isLoading: isLoadingGuards } = useNearbyGuards(
    currentLocation?.latitude ?? 0,
    currentLocation?.longitude ?? 0,
    5000, // 5km radius
  );

  const handleGuardPress = useCallback((guard: Guard) => {
    router.push(`/guards/${guard.id}`);
  }, []);

  const handleBookNow = () => router.push('/booking/create');
  const handleSchedule = () =>
    router.push({ pathname: '/booking/create', params: { type: 'scheduled' } });

  const handleMyLocation = () => {
```

```

    if (currentLocation) {
      mapRef.current?.animateToRegion({
        latitude: currentLocation.latitude,
        longitude: currentLocation.longitude,
        latitudeDelta: 0.01,
        longitudeDelta: 0.01,
      });
    }
  };

  const initialRegion: Region = {
    latitude: currentLocation?.latitude ?? 12.9716,
    longitude: currentLocation?.longitude ?? 77.5946,
    latitudeDelta: 0.05,
    longitudeDelta: 0.05,
  };

  if (activeBooking && activeBooking.status !== 'completed') {
    return (
      <View className="flex-1 bg-slate-900 items-center justify-center px-6">
        <Shield size={48} color="#3b82f6" />
        <Text className="text-white text-xl font-semibold mt-4 mb-2">
          Active Booking
        </Text>
        <Text className="text-slate-400 text-sm text-center mb-6">
          You have an ongoing booking with
          {activeBooking.guard?.name}
        </Text>
        <TouchableOpacity
          className="bg-blue-600 px-8 py-4 rounded-xl w-full items-center"
          onPress={() => router.push('/booking/tracking')}
        >
          <Text className="text-white font-semibold text-base">
            Track Guard
          </Text>
        </TouchableOpacity>
      </View>
    );
  }

  return (
    <View className="flex-1">
      {/* Full-screen map */}
      <MapView
        ref={mapRef}
        provider={PROVIDER_GOOGLE}
        style={{ flex: 1 }}
        initialRegion={initialRegion}
        showsUserLocation
        showsMyLocationButton={false}
        customMapStyle={darkMapStyle}

```



```

>
    {nearbyGuards?.map((guard) => (
      <MapGuardMarker
        key={guard.id}
        guard={guard}
        onPress={() => handleGuardPress(guard)}
      />
    ))}
  </MapView>

  {/* My Location button */}
  <TouchableOpacity
    onPress={handleMyLocation}
    className="absolute top-14 right-4 bg-slate-800 p-3
rounded-full shadow-lg"
  >
    <MapPin size={20} color="#3b82f6" />
  </TouchableOpacity>

  {/* Guards count badge */}
  {nearbyGuards && nearbyGuards.length > 0 && (
    <View className="absolute top-14 left-4 bg-slate-800 px-3
py-2 rounded-full flex-row items-center">
      <View className="w-2 h-2 bg-green-400 rounded-full
mr-2" />
      <Text className="text-white text-xs font-medium">
        {nearbyGuards.length} guards nearby
      </Text>
    </View>
  )}

  {/* Bottom Sheet */}
  <BottomSheet
    ref={bottomSheetRef}
    index={0}
    snapPoints={snapPoints}
    backgroundStyle={{ backgroundColor: '#1e293b' }}
    handleIndicatorStyle={{ backgroundColor: '#475569' }}
  >
    <BottomSheetScrollView
contentContainerStyle={{ paddingHorizontal: 16 }}>
      {/* CTA Buttons */}
      <View className="flex-row gap-3 mb-6 mt-2">
        <TouchableOpacity
          onPress={handleBookNow}
          className="flex-1 bg-blue-600 rounded-xl py-4 items-
center flex-row justify-center gap-2"
          activeOpacity={0.8}
        >
          <Shield size={18} color="#fff" />
          <Text className="text-white font-semibold text-
base">
            Book Now
          </Text>

```

```

        </TouchableOpacity>
        <TouchableOpacity
          onPress={handleSchedule}
          className="flex-1 bg-slate-700 rounded-xl py-4
items-center flex-row justify-center gap-2"
          activeOpacity={0.8}
        >
          <Clock size={18} color="#94a3b8" />
          <Text className="text-slate-300 font-semibold text-
base">
            Schedule
          </Text>
        </TouchableOpacity>
      </View>

      { /* Nearby Guards List */ }
      <Text className="text-white font-semibold text-base
mb-3">
        Nearby Guards
      </Text>

      {isLoadingGuards ? (
        <ActivityIndicator color="#3b82f6" className="mt-4" />
      ) : nearbyGuards?.length === 0 ? (
        <Text className="text-slate-400 text-sm text-center
mt-4">
          No guards available nearby right now.
        </Text>
      ) : (
        nearbyGuards?.map((guard) => (
          <GuardCard
            key={guard.id}
            guard={guard}
            onBook={() => handleGuardPress(guard)}
          />
        ))
      )}
    </BottomSheetScrollView>
  </BottomSheet>
</View>
);
}

// Google Maps dark style
const darkMapStyle = [
  { elementType: 'geometry', stylers: [{ color: '#212121' }] },
  { elementType: 'labels.icon', stylers: [{ visibility:
'off' }] },
  { elementType: 'labels.text.fill', stylers: [{ color:
'#757575' }] },
  { elementType: 'labels.text.stroke', stylers: [{ color:
'#212121' }] },
  {
    featureType: 'road',

```

```

        elementType: 'geometry',
        stylers: [{ color: '#383838' }],
      },
      {
        featureType: 'water',
        elementType: 'geometry',
        stylers: [{ color: '#000000' }],
      },
    ],
  ];

```

4. Guard Profile Screen

app/guards/[id].tsx

```

import {
  View,
  Text,
  Image,
  ScrollView,
  TouchableOpacity,
  ActivityIndicator,
} from 'react-native';
import { router, useLocalSearchParams } from 'expo-router';
import { useQuery } from '@tanstack/react-query';
import { guardService } from '@api/guardService';
import { RatingStars } from '@components/RatingStars';
import { LoadingOverlay } from '@components/LoadingOverlay';
import { GuardTier } from '@types';
import type { Review } from '@types';

const TIER_COLORS: Record<GuardTier, string> = {
  [GuardTier.BASIC]: '#64748b',
  [GuardTier.STANDARD]: '#3b82f6',
  [GuardTier.PREMIUM]: '#8b5cf6',
  [GuardTier.ELITE]: '#f59e0b',
};

export default function GuardProfileScreen() {
  const { id } = useLocalSearchParams<{ id: string }>();

  const { data: guard, isLoading: isLoadingGuard } = useQuery({
    queryKey: ['guard', id],
    queryFn: () => guardService.getGuardProfile(Number(id)),
    enabled: !!id,
  });

  const { data: reviews, isLoading: isLoadingReviews } =
    useQuery({
      queryKey: ['guard-reviews', id],
      queryFn: () => guardService.getGuardReviews(Number(id)),
      enabled: !!id,
    });

```

```

});

if (isLoadingGuard) return <LoadingOverlay />;
if (!guard) return null;

const tierColor = TIER_COLORS[guard.tier];

return (
  <View className="flex-1 bg-slate-900">
    <ScrollView>
      {/* Header */}
      <View className="relative">
        <View className="h-40 bg-gradient-to-b from-slate-800
to-slate-900" />
        <View className="absolute bottom-0 left-0 right-0 items-
center">
          <Image
            source={{
              uri: guard.profilePhoto ?? 'https://
via.placeholder.com/96',
            }}
            className="w-24 h-24 rounded-full border-4 border-
slate-900"
          />
        </View>
      </View>

      <View className="mt-14 px-6 items-center">
        <Text className="text-white text-2xl font-
bold">{guard.name}</Text>

        {/* Tier Badge */}
        <View
          className="px-3 py-1 rounded-full mt-2"
          style={{ backgroundColor: tierColor + '33' }}
        >
          <Text
            className="text-xs font-semibold capitalize"
            style={{ color: tierColor }}
          >
            {guard.tier} Guard
          </Text>
        </View>

        {/* Rating */}
        <View className="flex-row items-center mt-3 gap-2">
          <RatingStars rating={guard.rating} interactive={false}
size={18} />
          <Text className="text-slate-300 text-sm">
            {guard.rating.toFixed(1)} ({guard.totalReviews}
reviews)
          </Text>
        </View>
      </View>
    </ScrollView>
  </View>
);

```

```

    {/* Availability indicator */}
    <View className="flex-row items-center mt-2 gap-1">
      <View
        className={`w-2 h-2 rounded-full ${
          guard.isAvailable ? 'bg-green-400' : 'bg-red-400'
        }`}
      />
      <Text
        className={`text-xs ${
          guard.isAvailable ? 'text-green-400' : 'text-
red-400'
        }`}
      >
        {guard.isAvailable ? 'Available now' : 'Not
available'}
      </Text>
    </View>
  </View>

  {/* Bio */}
  {guard.bio ? (
    <View className="px-6 mt-6">
      <Text className="text-slate-400 text-sm
leading-5">{guard.bio}</Text>
    </View>
  ) : null}

  {/* Skills */}
  {guard.skills.length > 0 && (
    <View className="px-6 mt-6">
      <Text className="text-white font-semibold
mb-3">Skills</Text>
      <View className="flex-row flex-wrap gap-2">
        {guard.skills.map((skill) => (
          <View
            key={skill}
            className="bg-slate-700 px-3 py-1 rounded-full"
          >
            <Text className="text-slate-300 text-xs">{skill}
</Text>
          </View>
        ))}
      </View>
    </View>
  )}

  {/* Reviews */}
  <View className="px-6 mt-6 mb-32">
    <Text className="text-white font-semibold mb-3">
      Reviews ({reviews?.count ?? 0})
    </Text>
    {isLoadingReviews ? (
      <ActivityIndicator color="#3b82f6" />
    ) : reviews?.results.length === 0 ? (

```

```

        <Text className="text-slate-400 text-sm">No reviews
yet.</Text>
      ) : (
        reviews?.results.map((review: Review) => (
          <View key={review.id} className="bg-slate-800
rounded-xl p-4 mb-3">
            <View className="flex-row items-center gap-3
mb-2">
              <Image
                source={{
                  uri:
                    review.userPhoto ??
                    'https://via.placeholder.com/36',
                }}
                className="w-9 h-9 rounded-full"
              />
              <View className="flex-1">
                <Text className="text-white text-sm font-
medium">
                  {review.userName}
                </Text>
                <RatingStars
                  rating={review.rating}
                  interactive={false}
                  size={12}
                />
              </View>
            </View>
            <Text className="text-slate-400 text-sm
leading-5">
              {review.comment}
            </Text>
          </View>
        ))
      )}
    </View>
  </ScrollView>

  {/* Sticky Book CTA */}
  <View className="absolute bottom-0 left-0 right-0 bg-
slate-900 px-6 py-4 border-t border-slate-800">
    <TouchableOpacity
      onPress={() =>
        router.push({
          pathname: '/booking/create',
          params: { guardId: guard.id },
        })
      }
      disabled={!guard.isAvailable}
      className={`h-14 rounded-xl items-center justify-center
${
    guard.isAvailable ? 'bg-blue-600' : 'bg-slate-700'
}`}
      activeOpacity={0.8}
    </TouchableOpacity>
  </View>

```

```

    >
      <Text className="text-white font-semibold text-base">
        {guard.isAvailable ? 'Book this Guard' :
'Unavailable'}
      </Text>
    </TouchableOpacity>
  </View>
</View>
);
}

```

5. Create Booking Screen

app/booking/create.tsx

```

import { useState } from 'react';
import {
  View,
  Text,
  TextInput,
  TouchableOpacity,
  ScrollView,
  Alert,
  ActivityIndicator,
} from 'react-native';
import { router, useLocalSearchParams } from 'expo-router';
import { useCreateBooking } from '@/hooks/useCreateBooking';
import { useLocation } from '@/hooks/useLocation';
import { BookingType } from '@/types';
import type { BookingAddress, CreateBookingPayload } from '@/
types';

type Step = 'guard' | 'type' | 'address' | 'confirm';

const STEP_ORDER: Step[] = ['guard', 'type', 'address',
'confirm'];

export default function CreateBookingScreen() {
  const params = useLocalSearchParams<{ guardId?: string; type?:
string }>();

  const [step, setStep] = useState<Step>(
    params.guardId ? 'type' : 'guard',
  );
  const [guardId, setGuardId] = useState<number | null>(
    params.guardId ? Number(params.guardId) : null,
  );
  const [bookingType, setBookingType] = useState<BookingType>(
    params.type === 'scheduled' ? BookingType.SCHEDULED :
BookingType.ON_DEMAND,
  );

```

```

const [address, setAddress] = useState('');
const [latitude, setLatitude] = useState(0);
const [longitude, setLongitude] = useState(0);
const [notes, setNotes] = useState('');
const [scheduledAt, setScheduledAt] = useState('');

const { currentLocation } = useLocation();
const { createBooking, isCreating } = useCreateBooking();

const stepIndex = STEP_ORDER.indexOf(step);
const progress = ((stepIndex + 1) / STEP_ORDER.length) * 100;

const goNext = () => {
  const nextIndex = stepIndex + 1;
  if (nextIndex < STEP_ORDER.length) {
    setStep(STEP_ORDER[nextIndex]);
  }
};

const goBack = () => {
  if (stepIndex > 0) setStep(STEP_ORDER[stepIndex - 1]);
  else router.back();
};

const handleConfirm = async () => {
  if (!guardId) return;
  const payload: CreateBookingPayload = {
    guardId,
    type: bookingType,
    pickupAddress: {
      address,
      latitude: latitude || currentLocation?.latitude || 0,
      longitude: longitude || currentLocation?.longitude || 0,
    },
    notes: notes.trim() || undefined,
    scheduledAt:
      bookingType === BookingType.SCHEDULED ? scheduledAt :
undefined,
  };

  try {
    const booking = await createBooking(payload);
    router.replace('/booking/tracking');
  } catch (err: any) {
    Alert.alert('Booking Failed', err?.message ?? 'Please try
again.');
```



```

mb-4">
    <TouchableOpacity onPress={goBack}>
        <Text className="text-blue-400">
            {stepIndex === 0 ? 'Cancel' : '← Back'}
        </Text>
    </TouchableOpacity>
    <Text className="text-white font-semibold">New Booking</
Text>
    <Text className="text-slate-400 text-sm">
        {stepIndex + 1}/{STEP_ORDER.length}
    </Text>
</View>
{ /* Progress bar */ }
<View className="h-1 bg-slate-700 rounded-full">
    <View
        className="h-1 bg-blue-600 rounded-full"
        style={{ width: `${progress}%` }}
    />
</View>
</View>

<ScrollView className="flex-1 px-6 pt-6">
    { /* Step: Guard Selection */ }
    {step === 'guard' && (
        <View>
            <Text className="text-white text-xl font-semibold
mb-2">
                Select a Guard
            </Text>
            <Text className="text-slate-400 text-sm mb-6">
                Browse the map on the Home screen and tap a guard,
or enter a
                guard ID below.
            </Text>
            <View className="bg-slate-800 rounded-xl px-4 h-14
justify-center">
                <TextInput
                    className="text-white"
                    placeholder="Enter guard ID (temporary)"
                    placeholderTextColor="#475569"
                    keyboardType="number-pad"
                    onChangeText={(v) => setGuardId(v ? Number(v) :
null)}
                />
            </View>
            <TouchableOpacity
                onPress={goNext}
                disabled={!guardId}
                className={`mt-6 h-14 rounded-xl items-center
justify-center ${
                    guardId ? 'bg-blue-600' : 'bg-slate-700'
                }`}
            >
                <Text className="text-white font-

```

```

semibold">Continue</Text>
    </TouchableOpacity>
  </View>
)}

{/* Step: Booking Type */}
{step === 'type' && (
  <View>
    <Text className="text-white text-xl font-semibold
mb-6">
      When do you need the guard?
    </Text>
    {[
      {
        type: BookingType.ON_DEMAND,
        label: 'Right Now',
        desc: 'Guard arrives within 30 minutes',
      },
      {
        type: BookingType.SCHEDULED,
        label: 'Schedule Later',
        desc: 'Pick a date and time',
      },
    ]}.map(({ type, label, desc }) => (
      <TouchableOpacity
        key={type}
        onPress={() => setBookingType(type)}
        className={`p-4 rounded-xl mb-3 border-2 ${
          bookingType === type
            ? 'border-blue-600 bg-blue-600/10'
            : 'border-slate-700 bg-slate-800'
        }`}
      >
        <Text className="text-white font-semibold">{label}
      </Text>
        <Text className="text-slate-400 text-sm
mt-1">{desc}</Text>
      </TouchableOpacity>
    )})
  </View>
)}

{bookingType === BookingType.SCHEDULED && (
  <View className="bg-slate-800 rounded-xl px-4 h-14
justify-center mt-2">
    <TextInput
      className="text-white"
      placeholder="YYYY-MM-DD HH:MM"
      placeholderTextColor="#475569"
      value={scheduledAt}
      onChangeText={setScheduledAt}
    />
  </View>
)}

</TouchableOpacity>

```

```

        onPress={goNext}
        className="mt-6 h-14 bg-blue-600 rounded-xl items-
center justify-center"
      >
        <Text className="text-white font-
semibold">Continue</Text>
      </TouchableOpacity>
    </View>
  )}

  {/* Step: Address */}
  {step === 'address' && (
    <View>
      <Text className="text-white text-xl font-semibold
mb-2">
        Pickup Address
      </Text>
      <Text className="text-slate-400 text-sm mb-6">
        Where should the guard report to?
      </Text>

      <View className="bg-slate-800 rounded-xl px-4 py-3
mb-3 min-h-[80px]">
        <TextInput
          className="text-white"
          placeholder="Enter full address"
          placeholderTextColor="#475569"
          multiline
          numberOfLines={3}
          value={address}
          onChangeText={setAddress}
        />
      </View>

      {/* Use current location shortcut */}
      <TouchableOpacity
        onPress={() => {
          if (currentLocation) {
            setLatitude(currentLocation.latitude);
            setLongitude(currentLocation.longitude);
            setAddress('Current location');
          }
        }}
        className="flex-row items-center gap-2 mb-4"
      >
        <View className="w-2 h-2 bg-blue-400 rounded-full" /
>
        <Text className="text-blue-400 text-sm">Use my
current location</Text>
      </TouchableOpacity>

      {/* Notes */}
      <Text className="text-white font-medium mb-2">
        Notes (optional)

```

```

        </Text>
        <View className="bg-slate-800 rounded-xl px-4 py-3
min-h-[80px]">
            <TextInput
                className="text-white"
                placeholder="Any specific instructions for the
guard..."
                placeholderTextColor="#475569"
                multiline
                value={notes}
                onChangeText={setNotes}
            />
        </View>

        <TouchableOpacity
            onPress={goNext}
            disabled={!address.trim()}
            className={`mt-6 h-14 rounded-xl items-center
justify-center ${
                address.trim() ? 'bg-blue-600' : 'bg-slate-700'
            }`}
        >
            <Text className="text-white font-
semibold">Continue</Text>
        </TouchableOpacity>
    </View>
)}

{/* Step: Confirm */}
{step === 'confirm' && (
    <View>
        <Text className="text-white text-xl font-semibold
mb-6">
            Confirm Booking
        </Text>

        {[
            { label: 'Guard ID', value: String(guardId) },
            {
                label: 'Type',
                value: bookingType === BookingType.ON_DEMAND
                    ? 'On Demand'
                    : 'Scheduled',
            },
            { label: 'Pickup', value: address },
            ...(notes ? [{ label: 'Notes', value: notes }] :
[]),
            ...(scheduledAt ? [{ label: 'Scheduled At', value:
scheduledAt }] : []),
        ].map(({ label, value }) => (
            <View
                key={label}
                className="bg-slate-800 rounded-xl p-4 mb-3 flex-
row justify-between"

```

```

        >
        <Text className="text-slate-400 text-sm">{label}</
Text>
        <Text className="text-white text-sm font-medium
flex-1 text-right ml-4">
            {value}
        </Text>
    </View>
    )})

    {/* Price estimate */}
    <View className="bg-blue-600/10 border border-
blue-600/30 rounded-xl p-4 mt-2">
        <Text className="text-blue-400 text-sm font-medium">
            Estimated Cost
        </Text>
        <Text className="text-white text-2xl font-bold
mt-1">
            ₹499 – ₹999
        </Text>
        <Text className="text-slate-400 text-xs mt-1">
            Final amount billed based on actual duration
        </Text>
    </View>

    <TouchableOpacity
        onPress={handleConfirm}
        disabled={isCreating}
        className="mt-6 h-14 bg-blue-600 rounded-xl items-
center justify-center"
        activeOpacity={0.8}
    >
        {isCreating ? (
            <ActivityIndicator color="#fff" />
        ) : (
            <Text className="text-white font-semibold text-
base">
                Confirm Booking
            </Text>
        )}
    </TouchableOpacity>
</View>
    )}
    <View className="h-8" />
</ScrollView>
</View>
    );
}

```

6. Booking Tracking Screen

app/booking/tracking.tsx

```
import { useRef, useEffect } from 'react';
import {
  View,
  Text,
  TouchableOpacity,
  Alert,
  ScrollView,
} from 'react-native';
import MapView, { Marker, Polyline, PROVIDER_GOOGLE } from 'react-native-maps';
import { router } from 'expo-router';
import { Shield, AlertTriangle } from 'lucide-react-native';
import { useBookingStore } from '@store/useBookingStore';
import { useLocationStore } from '@store/useLocationStore';
import { useBookingWebSocket } from '@hooks/useBookingWebSocket';
import { StatusBadge } from '@components/StatusBadge';
import { BookingStatus } from '@types';

export default function BookingTrackingScreen() {
  const mapRef = useRef<MapView>(null);
  const { activeBooking, updateActiveBookingStatus } =
    useBookingStore();
  const { currentLocation } = useLocationStore();

  const { guardLocation, eta, isConnected } = useBookingWebSocket(
    activeBooking?.id ?? null,
  );

  // Pan map to show both user and guard
  useEffect(() => {
    if (guardLocation && currentLocation) {
      mapRef.current?.fitToCoordinates(
        [
          {
            latitude: currentLocation.latitude,
            longitude: currentLocation.longitude,
          },
          {
            latitude: guardLocation.latitude,
            longitude: guardLocation.longitude,
          },
        ],
        { edgePadding: { top: 80, right: 60, bottom: 300, left: 60 }, animated: true },
      );
    }
  }, [guardLocation]);

  const handleEndBooking = () => {
```

```

    Alert.alert('End Booking', 'Are you sure you want to end this
booking?', [
    { text: 'Cancel', style: 'cancel' },
    {
    text: 'End Booking',
    style: 'destructive',
    onPress: () => router.replace(`/booking/${
{activeBooking?.id}`),
    },
    ]);
};

const handleSOS = () => router.push('/sos');

if (!activeBooking) {
    router.replace('/(tabs)/home');
    return null;
}

const statusSteps = [
    { key: BookingStatus.PENDING, label: 'Booking Placed' },
    { key: BookingStatus.GUARD_ASSIGNED, label: 'Guard
Assigned' },
    { key: BookingStatus.GUARD_EN_ROUTE, label: 'Guard En
Route' },
    { key: BookingStatus.GUARD_ARRIVED, label: 'Guard Arrived' },
    { key: BookingStatus.ACTIVE, label: 'Service Active' },
];

const currentStepIndex = statusSteps.findIndex(
    (s) => s.key === activeBooking.status,
);

return (
    <View className="flex-1">
        {/* Map */}
        <MapView
            ref={mapRef}
            provider={PROVIDER_GOOGLE}
            style={{ flex: 1 }}
            showsUserLocation
            customMapStyle={darkMapStyle}
        >
            {guardLocation && (
                <Marker
                    coordinate={{
                        latitude: guardLocation.latitude,
                        longitude: guardLocation.longitude,
                    }}
                    title={activeBooking.guard?.name ?? 'Guard'}
                >
                    <View className="bg-blue-600 p-2 rounded-full">
                        <Shield size={16} color="#fff" />
                    </View>
                </Marker>
            )}
        </MapView>
    </View>
);

```

```

        </Marker>
    )}

    {/* Route line */}
    {guardLocation && currentLocation && (
        <Polyline
            coordinates={[
                {
                    latitude: guardLocation.latitude,
                    longitude: guardLocation.longitude,
                },
                {
                    latitude: currentLocation.latitude,
                    longitude: currentLocation.longitude,
                },
            ]}
            strokeColor="#3b82f6"
            strokeWidth={3}
            lineDashPattern={[8, 4]}
        />
    )}
</MapView>

{/* Connection indicator */}
<View className="absolute top-14 left-4 flex-row items-center bg-slate-900/80 px-3 py-2 rounded-full">
    <View
        className={`w-2 h-2 rounded-full mr-2 ${
            isConnected ? 'bg-green-400' : 'bg-red-400'
        }`}
    />
    <Text className="text-white text-xs">
        {isConnected ? 'Live tracking' : 'Reconnecting...'}
    </Text>
</View>

{/* SOS Button */}
<TouchableOpacity
    onPress={handleSOS}
    className="absolute top-14 right-4 bg-red-600 p-3 rounded-full shadow-lg"
>
    <AlertTriangle size={20} color="#fff" />
</TouchableOpacity>

{/* Bottom Panel */}
<View className="bg-slate-900 px-6 pt-5 pb-6 rounded-t-3xl shadow-2xl">
    {/* Guard info */}
    <View className="flex-row items-center justify-between mb-4">
        <View>
            <Text className="text-slate-400 text-xs mb-0.5">Your
Guard</Text>

```



```

        <Text className="text-white text-base font-semibold">
          {activeBooking.guard?.name ?? 'Assigning...'}
        </Text>
      </View>
    <View className="items-end">
      <StatusBadge status={activeBooking.status} />
      {eta != null && (
        <Text className="text-slate-400 text-xs mt-1">
          ETA {eta} min
        </Text>
      )}
    </View>
  </View>

  {/* Timeline */}
  <ScrollView
    horizontal
    showsHorizontalScrollIndicator={false}
    className="mb-5"
  >
    {statusSteps.map((step, index) => {
      const isDone = index <= currentStepIndex;
      const isCurrent = index === currentStepIndex;
      return (
        <View key={step.key} className="flex-row items-
center">
          <View className="items-center">
            <View
              className={`w-3 h-3 rounded-full ${
                isCurrent
                  ? 'bg-blue-500'
                  : isDone
                  ? 'bg-green-500'
                  : 'bg-slate-700'
              }`}
            />
            <Text
              className={`text-[10px] mt-1 w-16 text-center
${
                isCurrent ? 'text-blue-400' : isDone ?
'text-slate-300' : 'text-slate-600'
              }`}
            >
              {step.label}
            </Text>
          </View>
          {index < statusSteps.length - 1 && (
            <View
              className={`h-0.5 w-8 mb-4 ${
                index < currentStepIndex ? 'bg-green-500' :
'bg-slate-700'
              }`}
            />
          )}
        </View>
      )}
    )}
  </ScrollView>

```

```

        </View>
      );
    }
  }
</ScrollView>

  { /* End booking */
  {activeBooking.status === BookingStatus.ACTIVE && (
    <TouchableOpacity
      onPress={handleEndBooking}
      className="h-14 bg-red-600 rounded-xl items-center
justify-center"
      activeOpacity={0.8}
    >
      <Text className="text-white font-semibold text-base">
        End Booking
      </Text>
    </TouchableOpacity>
  )}
  </View>
</View>
);
}

const darkMapStyle = [
  { elementType: 'geometry', stylers: [{ color: '#212121' }] },
  { elementType: 'labels.text.fill', stylers: [{ color:
'#757575' }] },
  { featureType: 'road', elementType: 'geometry', stylers:
[ { color: '#383838' } ] },
  { featureType: 'water', elementType: 'geometry', stylers:
[ { color: '#000000' } ] },
];

```

7. Booking Detail Screen

app/booking/[id].tsx

```

import { useState } from 'react';
import {
  View,
  Text,
  ScrollView,
  TouchableOpacity,
  Alert,
  ActivityIndicator,
} from 'react-native';
import { useLocalSearchParams } from 'expo-router';
import { useQuery, useMutation, useQueryClient } from '@tanstack/
react-query';
import { bookingService } from '@api/bookingService';
import { reviewService } from '@api/reviewService';

```

```

import { RatingStars } from '@components/RatingStars';
import { StatusBadge } from '@components/StatusBadge';
import { LoadingOverlay } from '@components/LoadingOverlay';
import { BookingStatus } from '@types';
import { formatCurrency, formatDateTime } from '@utils/format';

export default function BookingDetailScreen() {
  const { id } = useLocalSearchParams<{ id: string }>();
  const qc = useQueryClient();

  const [rating, setRating] = useState(0);
  const [comment, setComment] = useState('');

  const { data: booking, isLoading } = useQuery({
    queryKey: ['booking', id],
    queryFn: () => bookingService.getBooking(Number(id)),
    enabled: !!id,
  });

  const { mutate: submitReview, isPending: isSubmitting } =
    useMutation({
      mutationFn: () =>
        reviewService.submitReview({
          bookingId: Number(id),
          rating,
          comment,
        }),
      onSuccess: () => {
        qc.invalidateQueries({ queryKey: ['booking', id] });
        Alert.alert('Thank you!', 'Your review has been
submitted.');
```

submitted.');

```

      },
      onError: (err: any) => {
        Alert.alert('Error', err?.message ?? 'Could not submit
review.');
```

review.');

```

      },
    });

  if (isLoading) return <LoadingOverlay />;
  if (!booking) return null;

  const isCompleted = booking.status === BookingStatus.COMPLETED;

  return (
    <ScrollView className="flex-1 bg-slate-900 px-6">
      <View className="pt-14 pb-6">
        <Text className="text-white text-2xl font-bold mb-1">
          Booking #{booking.id}
        </Text>
        <StatusBadge status={booking.status} />
      </View>

      { /* Guard Info */ }
      {booking.guard && (
```

```

        <View className="bg-slate-800 rounded-xl p-4 mb-4">
            <Text className="text-slate-400 text-xs mb-2 uppercase
tracking-wide">
                Guard
            </Text>
            <Text className="text-white font-semibold text-base">
                {booking.guard.name}
            </Text>
            <Text className="text-slate-400 text-sm mt-0.5
capitalize">
                {booking.guard.tier} tier
            </Text>
        </View>
    )}

    {/* Timeline */}
    <View className="bg-slate-800 rounded-xl p-4 mb-4">
        <Text className="text-slate-400 text-xs mb-3 uppercase
tracking-wide">
            Timeline
        </Text>
        {booking.timeline.map((event, i) => (
            <View key={i} className="flex-row items-start mb-3">
                <View className="w-2 h-2 bg-blue-500 rounded-full
mt-1.5 mr-3" />
                <View className="flex-1">
                    <Text className="text-white text-sm capitalize">
                        {event.status.replace(/_/g, ' ')}
                    </Text>
                    <Text className="text-slate-500 text-xs mt-0.5">
                        {formatDateTime(event.timestamp)}
                    </Text>
                    {event.note && (
                        <Text className="text-slate-400 text-xs mt-0.5">
                            {event.note}
                        </Text>
                    )}
                </View>
            </View>
        )})}
    </View>

    {/* Amount Breakdown */}
    <View className="bg-slate-800 rounded-xl p-4 mb-4">
        <Text className="text-slate-400 text-xs mb-3 uppercase
tracking-wide">
            Amount Breakdown
        </Text>
        {[
            {
                label: 'Duration',
                value: booking.durationMinutes
                ? `${booking.durationMinutes} min`
                : '-',
            }
        ]}
    </View>

```

```

    },
    {
      label: 'Rate',
      value: formatCurrency(booking.ratePerHour) + '/hr',
    },
    {
      label: 'Platform Fee',
      value: formatCurrency(booking.platformFee),
    },
    {
      label: 'Paid from Wallet',
      value: `-${formatCurrency(booking.paidFromWallet)}`,
    },
  ].map(({ label, value }) => (
    <View key={label} className="flex-row justify-between
mb-2">
      <Text className="text-slate-400 text-sm">{label}</
Text>
      <Text className="text-white text-sm">{value}</Text>
    </View>
  )))
  <View className="border-t border-slate-700 mt-2 pt-2 flex-
row justify-between">
    <Text className="text-white font-semibold">Total</Text>
    <Text className="text-white font-bold text-base">
      {formatCurrency(booking.totalAmount)}
    </Text>
  </View>
</View>

{/* Rate & Review */}
{isCompleted && (
  <View className="bg-slate-800 rounded-xl p-4 mb-8">
    <Text className="text-white font-semibold mb-3">
      Rate your experience
    </Text>
    <RatingStars
      rating={rating}
      interactive
      size={32}
      onRate={setRating}
    />
    <View className="bg-slate-700 rounded-xl px-4 py-3 mt-4
min-h-[80px]">
      <Text
        // @ts-ignore
        className="text-white"
        onChangeText={setComment}
        placeholder="Share your experience..."
      />
    </View>
    <TouchableOpacity
      onPress={() => submitReview()}
      disabled={rating === 0 || isSubmitting}

```

```

        className={`mt-4 h-12 rounded-xl items-center justify-
center ${
        rating > 0 ? 'bg-blue-600' : 'bg-slate-700'
    }}`
    >
        {isSubmitting ? (
            <ActivityIndicator color="#fff" />
        ) : (
            <Text className="text-white font-semibold">Submit
Review</Text>
        )}
    </TouchableOpacity>
</View>
)}
</ScrollView>
);
}

```

8. Bookings Tab

app/(tabs)/bookings.tsx

```

import { useState } from 'react';
import {
    View,
    Text,
    FlatList,
    TouchableOpacity,
    ActivityIndicator,
    RefreshControl,
} from 'react-native';
import { useQuery } from '@tanstack/react-query';
import { router } from 'expo-router';
import { bookingService } from '@api/bookingService';
import { BookingCard } from '@components/BookingCard';
import { BookingStatus } from '@types';
import type { Booking } from '@types';

type Tab = 'active' | 'history';

const ACTIVE_STATUSES: BookingStatus[] = [
    BookingStatus.PENDING,
    BookingStatus.GUARD_ASSIGNED,
    BookingStatus.GUARD_EN_ROUTE,
    BookingStatus.GUARD_ARRIVED,
    BookingStatus.ACTIVE,
];

export default function BookingsTab() {
    const [activeTab, setActiveTab] = useState<Tab>('active');

```

```

const {
  data: bookings,
  isLoading,
  refetch,
  isRefetching,
} = useQuery({
  queryKey: ['bookings', activeTab],
  queryFn: () => bookingService.getBookingList({ tab:
activeTab }),
  staleTime: 30_000,
});

const renderItem = ({ item }: { item: Booking }) => (
  <BookingCard
    booking={item}
    onPress={() => {
      if (
        ACTIVE_STATUSES.includes(item.status) &&
        item.status !== BookingStatus.PENDING
      ) {
        router.push('/booking/tracking');
      } else {
        router.push(`/booking/${item.id}`);
      }
    }}
  />
);

return (
  <View className="flex-1 bg-slate-900">
    { /* Header */ }
    <View className="px-6 pt-14 pb-4 bg-slate-900">
      <Text className="text-white text-2xl font-bold
mb-4">Bookings</Text>
      { /* Tabs */ }
      <View className="flex-row bg-slate-800 rounded-xl p-1">
        {[ 'active', 'history' ] as Tab[] }.map((tab) => (
          <TouchableOpacity
            key={tab}
            onPress={() => setActiveTab(tab)}
            className={`flex-1 py-2.5 rounded-lg items-center ${
              activeTab === tab ? 'bg-slate-600' : ''
            }`}
          >
            <Text
              className={`font-medium text-sm capitalize ${
                activeTab === tab ? 'text-white' : 'text-
slate-400'
              }`}
            />
            {tab}
          </Text>
        </TouchableOpacity>
      ))}
    </View>
  </View>
);

```

```

    </View>
  </View>

  {isLoading ? (
    <ActivityIndicator color="#3b82f6" className="mt-8" />
  ) : (
    <FlatList
      data={bookings?.results ?? []}
      renderItem={renderItem}
      keyExtractor={(item) => String(item.id)}
      contentContainerStyle={{ paddingHorizontal: 24,
paddingBottom: 32 }}
      ItemSeparatorComponent={() => <View className="h-3" />}
      refreshControl={
        <RefreshControl
          refreshing={isRefetching}
          onRefresh={refetch}
          tint-color="#3b82f6"
        />
      }
      ListEmptyComponent={
        <View className="items-center mt-16">
          <Text className="text-slate-400 text-base">
            No {activeTab === 'active' ? 'active' : 'past'}
bookings
          </Text>
        </View>
      }
    />
  )}
</View>
);
}

```

9. Wallet Tab

app/(tabs)/wallet.tsx

```

import {
  View,
  Text,
  FlatList,
  TouchableOpacity,
  ActivityIndicator,
  Alert,
} from 'react-native';
import { Plus, ArrowDownLeft, ArrowUpRight } from 'lucide-react-native';
import { useWallet } from '@/hooks/useWallet';
import { useRazorpay } from '@/hooks/useRazorpay';
import { TransactionType } from '@/types';

```



```

import { formatCurrency, formatDateTime } from '@utils/format';
import type { Transaction } from '@types';

const TOP_UP_AMOUNTS = [200, 500, 1000, 2000];

export default function WalletTab() {
  const { wallet, transactions, isLoading, refetch } =
    useWallet();
  const { openCheckout, isProcessing } = useRazorpay(0);

  const handleTopUp = (amount: number) => {
    Alert.alert(`Add ₹${amount}`, `Top up your wallet with ₹$
{amount}?', [
      { text: 'Cancel', style: 'cancel' },
      {
        text: 'Proceed',
        onPress: () => openCheckout(amount),
      },
    ]);
  };

  const renderTransaction = ({ item }: { item: Transaction }) => {
    const isCredit = item.type === TransactionType.CREDIT;
    return (
      <View className="flex-row items-center bg-slate-800 rounded-
xl p-4">
        <View
          className={`w-10 h-10 rounded-full items-center justify-
center mr-3 ${
            isCredit ? 'bg-green-500/20' : 'bg-red-500/20'
          }}`>
          >
            {isCredit ? (
              <ArrowDownLeft size={18} color="#22c55e" />
            ) : (
              <ArrowUpRight size={18} color="#ef4444" />
            )}
          </View>
          <View className="flex-1">
            <Text className="text-white text-sm font-medium">
              {item.description}
            </Text>
            <Text className="text-slate-500 text-xs mt-0.5">
              {formatDateTime(item.createdAt)}
            </Text>
          </View>
          <Text
            className={`font-semibold text-sm ${
              isCredit ? 'text-green-400' : 'text-red-400'
            }}`>
            >
              {isCredit ? '+' : '-'}
              {formatCurrency(item.amount)}
            </Text>

```

```

        </View>
    );
};

return (
    <View className="flex-1 bg-slate-900">
        <FlatList
            data={transactions?.results ?? []}
            renderItem={renderTransaction}
            keyExtractor={({item}) => String(item.id)}
            ListHeaderComponent={
                <View className="px-6 pt-14 pb-4">
                    <Text className="text-white text-2xl font-bold
mb-6">Wallet</Text>

                    {/* Balance Card */}
                    <View className="bg-gradient-to-br from-blue-600 to-
blue-800 rounded-2xl p-6 mb-6">
                        <Text className="text-blue-200 text-sm mb-1">
                            Available Balance
                        </Text>
                        <isLoading ? (
                            <ActivityIndicator color="#fff" />
                        ) : (
                            <Text className="text-white text-4xl font-bold">
                                {formatCurrency(wallet?.balance ?? 0)}
                            </Text>
                        )}
                    </View>

                    {/* Top-up amounts */}
                    <Text className="text-white font-semibold mb-3">
                        Add Money
                    </Text>
                    <View className="flex-row flex-wrap gap-3 mb-6">
                        {TOP_UP_AMOUNTS.map((amount) => (
                            <TouchableOpacity
                                key={amount}
                                onPress={() => handleTopUp(amount)}
                                disabled={isProcessing}
                                className="flex-row items-center gap-1.5 bg-
slate-800 border border-slate-700 px-4 py-3 rounded-xl"
                            >
                                <Plus size={14} color="#3b82f6" />
                                <Text className="text-white text-sm font-
medium">
                                    ₹{amount}
                                </Text>
                            </TouchableOpacity>
                        )})}
                    </View>

                    <Text className="text-white font-semibold mb-3">
                        Transactions

```

```

        </Text>
      </View>
    }
    ItemSeparatorComponent={() => <View className="h-2 mx-6" /
  >}
  contentContainerStyle={{ paddingBottom: 32,
paddingHorizontal: 24 }}
  ListEmptyComponent={
    !isLoading ? (
      <View className="items-center mt-6">
        <Text className="text-slate-400 text-sm">
          No transactions yet.
        </Text>
      </View>
    ) : null
  }
/>
</View>
);
}

```

10. Profile Tab

app/(tabs)/profile.tsx

```

import { useState } from 'react';
import {
  View,
  Text,
  Image,
  TextInput,
  TouchableOpacity,
  ScrollView,
  Alert,
  ActivityIndicator,
} from 'react-native';
import { useMutation, useQueryClient } from '@tanstack/react-
query';
import { router } from 'expo-router';
import {
  Logout,
  ChevronRight,
  Shield,
  Bell,
  HelpCircle,
  FileText,
} from 'lucide-react-native';
import { useAuthStore } from '@store/useAuthStore';
import { userService } from '@api/userService';

const MENU_ITEMS = [

```

```

    { icon: Bell, label: 'Notifications', route: '/notifications' },
    { icon: Shield, label: 'Privacy & Security', route: '/
privacy' },
    { icon: HelpCircle, label: 'Help & Support', route: '/
support' },
    { icon: FileText, label: 'Terms & Conditions', route: '/
terms' },
  ] as const;

export default function ProfileTab() {
  const { user, logout, setUser } = useAuthStore();
  const qc = useQueryClient();

  const [isEditing, setIsEditing] = useState(false);
  const [name, setName] = useState(user?.name ?? '');
  const [email, setEmail] = useState(user?.email ?? '');

  const { mutate: updateProfile, isPending } = useMutation({
    mutationFn: () => userService.updateProfile({ name, email }),
    onSuccess: (updated) => {
      setUser(updated);
      setIsEditing(false);
    },
    onError: (err: any) => {
      Alert.alert('Error', err?.message ?? 'Could not update
profile.');
```

profile.');

```

    },
  });

  const handleLogout = () => {
    Alert.alert('Log Out', 'Are you sure you want to log out?', [
      { text: 'Cancel', style: 'cancel' },
      {
        text: 'Log Out',
        style: 'destructive',
        onPress: async () => {
          await logout();
          qc.clear();
          router.replace('/(auth)/login');
        },
      },
    ],
  );
};

return (
  <ScrollView className="flex-1 bg-slate-900">
    <View className="px-6 pt-14 pb-8">
      { /* Avatar */ }
      <View className="items-center mb-6">
        <Image
          source={{
            uri: user?.profilePhoto ?? 'https://
via.placeholder.com/80',
          }}

```

```

        className="w-20 h-20 rounded-full border-2 border-
slate-700 mb-3"
      />
      {!isEditing ? (
        <>
          <Text className="text-white text-xl font-
bold">{user?.name}</Text>
          <Text className="text-slate-400 text-sm
mt-0.5">{user?.phone}</Text>
          {user?.email && (
            <Text className="text-slate-400 text-
sm">{user.email}</Text>
          )}
          <TouchableOpacity
            onPress={() => setIsEditing(true)}
            className="mt-3 bg-slate-700 px-5 py-2 rounded-
full"
          >
            <Text className="text-white text-sm">Edit
Profile</Text>
          </TouchableOpacity>
        </>
      ) : (
        <View className="w-full mt-2">
          <View className="bg-slate-800 rounded-xl px-4 h-12
justify-center mb-3">
            <TextInput
              className="text-white"
              value={name}
              onChangeText={setName}
              placeholder="Full name"
              placeholderTextColor="#475569"
            />
          </View>
          <View className="bg-slate-800 rounded-xl px-4 h-12
justify-center mb-4">
            <TextInput
              className="text-white"
              value={email}
              onChangeText={setEmail}
              placeholder="Email (optional)"
              placeholderTextColor="#475569"
              keyboardType="email-address"
              autoCapitalize="none"
            />
          </View>
          <View className="flex-row gap-3">
            <TouchableOpacity
              onPress={() => setIsEditing(false)}
              className="flex-1 h-12 bg-slate-700 rounded-xl
items-center justify-center"
            >
              <Text className="text-white text-sm">Cancel</
Text>

```

```

        </TouchableOpacity>
        <TouchableOpacity
            onPress={() => updateProfile()}
            disabled={isPending}
            className="flex-1 h-12 bg-blue-600 rounded-xl
items-center justify-center"
        >
            {isPending ? (
                <ActivityIndicator color="#fff" size="small" /
>
                ) : (
                <Text className="text-white text-sm font-
semibold">Save</Text>
                )}
        </TouchableOpacity>
    </View>
</View>
)}
</View>

{/* Menu Items */}
<View className="bg-slate-800 rounded-2xl overflow-hidden
mb-4">
    {MENU_ITEMS.map(({ icon: Icon, label, route }, index) =>
    (
        <TouchableOpacity
            key={route}
            onPress={() => router.push(route as any)}
            className={`flex-row items-center px-4 py-4 ${
                index < MENU_ITEMS.length - 1
                ? 'border-b border-slate-700'
                : ''
            }`}
        >
            <Icon size={18} color="#64748b" />
            <Text className="text-white text-sm flex-1
ml-3">{label}</Text>
            <ChevronRight size={16} color="#475569" />
        </TouchableOpacity>
    )))
</View>

{/* Logout */}
<TouchableOpacity
    onPress={handleLogout}
    className="flex-row items-center justify-center bg-
red-600/10 border border-red-600/30 rounded-xl py-4 gap-2"
>
    <LogOut size={18} color="#ef4444" />
    <Text className="text-red-400 font-semibold">Log Out</
Text>
</TouchableOpacity>

    <Text className="text-slate-600 text-xs text-center mt-6">

```

```

        b-secure v1.0.0
      </Text>
    </View>
  </ScrollView>
);
}

```

11. SOS Screen

app/sos.tsx

```

import { useState, useRef, useCallback } from 'react';
import {
  View,
  Text,
  TouchableOpacity,
  Animated,
  Alert,
} from 'react-native';
import { router } from 'expo-router';
import * as Haptics from 'expo-haptics';
import { useBookingStore } from '@store/useBookingStore';
import { useLocationStore } from '@store/useLocationStore';
import { useSOSTrigger } from '@hooks/useSOSTrigger';

const HOLD_DURATION = 3000; // 3 seconds

export default function SOSScreen() {
  const [isHolding, setIsHolding] = useState(false);
  const [triggered, setTriggered] = useState(false);
  const progressAnim = useRef(new Animated.Value(0)).current;
  const holdAnimation = useRef<Animated.CompositeAnimation | null>(null);
  const holdTimerRef = useRef<ReturnType<typeof setTimeout> | null>(null);

  const { activeBooking } = useBookingStore();
  const { currentLocation } = useLocationStore();
  const { triggerSOS, isTriggering } =
    useSOSTrigger(activeBooking?.id ?? null);

  const scaleAnim = useRef(new Animated.Value(1)).current;

  const pulse = useRef(
    Animated.loop(
      Animated.sequence([
        Animated.timing(scaleAnim, {
          toValue: 1.08,
          duration: 600,
          useNativeDriver: true,
        }),
      ]),
    ),
  );

```

```

        Animated.timing(scaleAnim, {
          toValue: 1,
          duration: 600,
          useNativeDriver: true,
        }),
      ],
    ),
  ).current;

const startHold = useCallback(() => {
  setIsHolding(true);
  Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Heavy);
  pulse.start();

  holdAnimation.current = Animated.timing(progressAnim, {
    toValue: 1,
    duration: HOLD_DURATION,
    useNativeDriver: false,
  });
  holdAnimation.current.start();

  holdTimerRef.current = setTimeout(async () => {
    pulse.stop();
    scaleAnim.setValue(1);

    Haptics.notificationAsync(Haptics.NotificationFeedbackType.Warning);

    try {
      await triggerSOS({
        latitude: currentLocation?.latitude ?? 0,
        longitude: currentLocation?.longitude ?? 0,
      });
      setTriggered(true);

      Haptics.notificationAsync(Haptics.NotificationFeedbackType.Success);
    } catch (err: any) {
      Alert.alert('SOS Failed', err?.message ?? 'Could not send SOS. Call 112.');
```

cancelHold();

```

    }
  }, HOLD_DURATION);
}, [currentLocation]);

const cancelHold = useCallback(() => {
  holdAnimation.current?.stop();
  holdTimerRef.current && clearTimeout(holdTimerRef.current);
  pulse.stop();
  scaleAnim.setValue(1);
  progressAnim.setValue(0);
  setIsHolding(false);
  Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Light);
}, []);

const progressDeg = progressAnim.interpolate({

```



```

    inputRange: [0, 1],
    outputRange: ['0deg', '360deg'],
  });

  if (triggered) {
    return (
      <View className="flex-1 bg-red-950 items-center justify-
center px-6">
        <View className="w-24 h-24 bg-red-600 rounded-full items-
center justify-center mb-6">
          <Text className="text-white text-4xl">!</Text>
        </View>
        <Text className="text-white text-2xl font-bold mb-2 text-
center">
          SOS Sent
        </Text>
        <Text className="text-red-300 text-sm text-center
leading-6 mb-8">
          Emergency services and our response team have been
alerted.
          Help is on the way.
        </Text>
        <TouchableOpacity
          onPress={() => router.back()}
          className="bg-white/10 border border-white/20 px-8 py-4
rounded-xl"
        >
          <Text className="text-white font-semibold">Go Back</
Text>
        </TouchableOpacity>
      </View>
    );
  }

  return (
    <View className="flex-1 bg-slate-900 items-center justify-
center px-6">
      {/* Cancel */}
      <TouchableOpacity
        onPress={() => router.back()}
        className="absolute top-14 left-6"
      >
        <Text className="text-slate-400">Cancel</Text>
      </TouchableOpacity>

      <Text className="text-white text-2xl font-bold
mb-2">Emergency SOS</Text>
      <Text className="text-slate-400 text-sm text-center mb-12
leading-6">
        Hold the button for 3 seconds to send an SOS alert to our
response team
        and emergency services.
      </Text>
    </View>
  );
}

```

```

    {/ * SOS Button with radial progress */}
    <View className="relative items-center justify-center">
      {/ * Outer ring progress indicator */}
      <Animated.View
        style={{
          position: 'absolute',
          width: 200,
          height: 200,
          borderRadius: 100,
          borderWidth: 4,
          borderColor: '#ef4444',
          opacity: progressAnim.interpolate({
            inputRange: [0, 0.01, 1],
            outputRange: [0.2, 0.8, 1],
          }),
          transform: [{ scale: scaleAnim }],
        }}
      />

      <Animated.View style={{ transform: [{ scale:
scaleAnim }] }}>
        <TouchableOpacity
          onPressIn={startHold}
          onPressOut={cancelHold}
          activeOpacity={1}
          style={{
            width: 160,
            height: 160,
            borderRadius: 80,
            backgroundColor: '#dc2626',
            alignItems: 'center',
            justifyContent: 'center',
            shadowColor: '#ef4444',
            shadowOpacity: 0.6,
            shadowRadius: 20,
            elevation: 12,
          }}
        >
          <Text className="text-white text-5xl font-black">SOS</
Text>
          </TouchableOpacity>
        </Animated.View>
      </View>

      {isHolding && (
        <Text className="text-red-400 text-sm font-medium mt-8">
          Keep holding...
        </Text>
      )}

      <Text className="text-slate-600 text-xs text-center mt-12">
        For immediate help, also call{' '}
        <Text className="text-red-400 font-semibold">112</Text>
      </Text>

```

```
        </View>
    );
}
```